

STUDY GUIDE

From First Principles to Cross-Feature Fusion Mamba Networks

*Deep learning, detection, state-space models,
visible–thermal fusion and tracking,
in the order this project needs them*

Ritesh Roshan

Version 1.0
October 2026

Contents

1	How to use this guide	1
1.1	The five stages	2
1.2	How each chapter is organised	2
1.3	How to study	2
2	Stage one: foundations	3
2.1	Why it matters	3
2.2	What to learn	3
2.3	Lectures and reading	4
2.4	Papers	5
2.5	In this repository	5
2.6	Exercises	5
2.7	You are ready when	6
3	Stage two: detection and YOLO	7
3.1	Why it matters	7
3.2	What to learn	7
3.3	Lectures and reading	8
3.4	Papers	9
3.5	In this repository	9
3.6	Exercises	9
3.7	You are ready when	10
4	Stage three: state-space models and Mamba	11
4.1	Why it matters	11
4.2	The one equation to own	11
4.3	What to learn	12
4.4	Lectures and reading	12
4.5	Papers	13
4.6	In this repository	13
4.7	Exercises	13
4.8	You are ready when	13

5	Stage four: seeing in two spectra	14
5.1	Why it matters	14
5.2	What to learn	14
5.3	Papers	15
5.4	In this repository	15
5.5	Exercises	15
5.6	You are ready when	16
6	Stage five: tracking and video	17
6.1	Why it matters	17
6.2	What to learn	17
6.3	Lectures and reading	18
6.4	Papers	18
6.5	In this repository	18
6.6	Exercises	19
6.7	You are ready when	19
7	From study to research	20
7.1	Reading the project's own results	20
7.2	Two lessons from the training logs	20
7.3	A weekly rhythm	21
7.4	When you are stuck	21
A	Glossary	22

How to use this guide

CFFM-Net is an object detector that looks through two cameras at once, an ordinary visible camera and a thermal camera, and decides at every location which one to trust. Understanding it end to end touches five areas that are usually taught separately: deep learning for images, object detection, state-space models, multispectral fusion and multi-object tracking. This guide puts them in the order the project needs them and, for each one, names the lectures to watch, the papers to read, the files in the repository that use the idea, and a short set of exercises.

The path assumes comfortable Python and first-year linear algebra and calculus. It does not assume prior deep learning. Part time, it takes about two months; most people can start contributing to the code after the second stage.

1.1 The five stages

	stage	time	what it unlocks in CFFM-Net
1	Foundations	3–4 weeks	training loops, convolutions, residual networks, attention
2	Detection and YOLO	2 weeks	the YOLO26 backbone, neck and NMS-free head the method is built on
3	State-space models and Mamba	2 weeks	the selective scan and the reliability gate on its step size
4	Seeing in two spectra	1 week	paired visible–thermal data, fusion designs, the datasets
5	Tracking and video	1 week	identities over time, the showcase tracker, the next phase

1.2 How each chapter is organised

Every stage chapter has the same six parts. *Why it matters* says what breaks in the project without it. *What to learn* lists the ideas to own, not just recognise. *Lectures and reading* gives the resources, with links. *Papers* lists the PDFs in study-prep/papers, each with the one thing to take from it. *In this repository* names the code to read once the stage is done. *Exercises* are small and concrete; the chapter ends with a test of readiness.

1.3 How to study

Watch a lecture once at normal speed, then work the matching assignment or exercise before moving on; the assignments teach more than the videos. Read a paper three times: first the abstract, figures and conclusion; then the method, with a pen, rederiving each equation; then the experiments, asking what was compared, against what, and whether the comparison is fair. Keep a notebook of every idea in your own words. When a paper and a lecture disagree, the code decides.

Stage one: foundations

2.1 Why it matters

Every part of CFFM-Net is a neural network trained by gradient descent. Without a working mental model of how a loss produces gradients, how an optimiser uses them and why normalisation and residual connections make deep networks trainable, the rest of the project is a set of incantations. This stage builds that model from scratch.

2.2 What to learn

- Linear classifiers, softmax and cross-entropy; what a loss surface is.
- Backpropagation as the chain rule on a computation graph, implemented by hand once.
- Stochastic gradient descent, momentum and Adam [1]; learning-rate schedules and warm-up.
- Convolutions, padding, stride and receptive fields; pooling; feature maps as images of features.
- Batch normalisation [2] and why it stabilises training.
- Residual connections [3]: why a block that starts as the identity is easy to optimise. CFFM-Net uses the same trick, starting every fusion block as an average.
- Overfitting, data augmentation, train, validation and test splits.
- Attention and the Transformer [4], and its application to images [5].
- Mixed precision: what float16 can and cannot represent. The project hit this directly, as Chapter 7 explains.

2.3 Lectures and reading

resource	link and what to take from it
Stanford CS231n, course site and notes	https://cs231n.stanford.edu/ https://cs231n.github.io/ The notes on optimisation, backpropagation and convolutional networks are the core text.
CS231n lectures, Spring 2017	https://www.youtube.com/playlist?list=PL3FW7Lu3i5JvHM8ljYj-zLfQRF3EO8sYv Lectures 1 to 11. Do assignments 1 and 2.
Neural Networks: Zero to Hero, Andrej Karpathy	https://karpathy.ai/zero-to-hero.html Builds backpropagation and a training loop from nothing; watch alongside CS231n.
Deep Learning for Computer Vision, Michigan EECS 498	https://www.youtube.com/playlist?list=PL5-TkQAFaZFbzxjBHtzdVCWE0Zbhong7r A more recent course by Justin Johnson, with an excellent detection lecture.
3Blue1Brown, neural networks	https://www.3blue1brown.com/topics/neural-networks Geometric intuition for gradient descent and backpropagation.
Dive into Deep Learning	https://d2l.ai/ A free book with runnable code for every chapter; use it as a reference.

2.4 Papers

paper	take away
ResNet [3]	identity shortcuts make very deep networks trainable
Batch Normalization [2]	normalising activations stabilises and speeds up training
Adam [1]	per-parameter adaptive step sizes; the default optimiser here
Attention Is All You Need [4]	attention as content-based mixing of a sequence
ViT [5]	images as sequences of patches; the bridge to sequence models for vision

2.5 In this repository

Read `cffm-net-pilot/src/cffm/scan.py`. The reference scan is plain PyTorch and makes a good first real code read: a recurrence, a chunked parallel version of it, and a test that the two agree.

2.6 Exercises

1. Implement a two-layer network and its backward pass in NumPy; check every gradient numerically.
2. Train a small convolutional network on CIFAR-10 and plot training and validation loss. Make it overfit on purpose, then fix it with augmentation.
3. Remove the shortcut from a twenty-layer residual network and compare the training curves.
4. In float16, compute 0.9^{1000} by repeated multiplication. Explain the result.

2.7 You are ready when

You can explain, without notes, why a residual block that starts as the identity trains more easily than a plain block, and you can write a correct training loop for an image classifier from an empty file.

Stage two: detection and YOLO

3.1 Why it matters

CFFM-Net is a detector first. Its backbones, neck, head, loss and evaluation all come from YOLO26, and every result in the project is a COCO average precision. To change the method, or even to read its results tables critically, you need to know how a detector turns a feature map into boxes and how those boxes are scored.

3.2 What to learn

- Two-stage detection (region proposals, then classification) and one-stage detection; why YOLO is fast [6, 7].
- Box regression, anchors, and anchor-free heads.
- Feature pyramids [8]: why small objects need high-resolution features, and what a stride-4 head buys and costs.
- Class imbalance and the focal loss [9].
- Non-maximum suppression, and why DETR [10] could remove it with one-to-one matching. YOLO26 and YOLOv10 [11] borrow this idea; the real-time DETR line [12] is the main competitor.
- Evaluation: intersection over union, precision and recall, AP50 and AP50-95, and the size bands for small and tiny objects.

3.3 Lectures and reading

resource	link and what to take from it
CS231n lecture on detection and segmentation	https://www.youtube.com/playlist?list=PL3FW7Lu3i5JvHM8ljYj-zLfQRF3EO8sYv Lecture 11 sets out the whole landscape.
EECS 498 lectures on object detection	https://www.youtube.com/playlist?list=PL5-TkQAfAZFbzxjBHtzdVCWE0Zbhong7r Two lectures that go deeper than CS231n on single-stage detectors.
Ultralytics documentation	https://docs.ultralytics.com/ The trainer, validator and data pipeline this project subclasses.
YOLO26 model page	https://docs.ultralytics.com/models/yolo26/ The architecture and end-to-end head CFFM-Net builds on.
The Illustrated Transformer	https://jalammar.github.io/illustrated-transformer/ Read before DETR.

3.4 Papers

paper	take away
Faster R-CNN [6]	the two-stage design every later paper compares against
YOLOv1 [7]	detection as one regression over a grid
FPN [8]	combine coarse semantics with fine resolution across scales
RetinaNet [9]	the focal loss for the flood of easy background
YOLOv3 [13]	multi-scale prediction in the YOLO family
DETR [10]	set prediction with one-to-one matching, no NMS
RT-DETR [12]	real-time transformers that rival YOLO
YOLOv10 [11]	NMS-free training for YOLO, the ancestor of YOLO26's head

3.5 In this repository

Read `cfm-net-pilot/src/cfm/model.py`, which wraps YOLO26 into a two-stream model, and `train.py`, which adds the size-binned evaluation. Then open notebook 04 and compare its single-camera results with the two-stream ones in notebook 05.

3.6 Exercises

1. Implement IoU and greedy NMS for a list of boxes; test them on hand-made cases.
2. Fine-tune YOLO26-n on a small public dataset with the Ultralytics CLI and read every line of its output.
3. Compute AP50 for one class by hand from a list of scored detections and ground truths.
4. Explain in a paragraph why the stride-4 head cost the pilot 1.8 AP on LLVIP but might help on M3FD.

3.7 You are ready when

You can draw a YOLO-style detector from input image to final boxes, name what each part is for, and explain the difference between AP50 and AP50-95 with an example.

Stage three: state-space models and Mamba

4.1 Why it matters

This is the stage that standard computer-vision courses do not cover, and it is the heart of CFFM-Net. The fusion block interleaves visible and thermal features into one sequence and runs a selective state-space scan over it. The project’s central idea is a single multiplication inside that scan, so the scan itself has to be understood exactly.

4.2 The one equation to own

A selective state-space model keeps a small hidden state h per channel and walks along the sequence:

$$h_k = \exp(\Delta_k A) h_{k-1} + \Delta_k B_k x_k, \quad y_k = C_k h_k + D x_k.$$

A is a learned negative diagonal, so $\exp(\Delta_k A)$ is a number between zero and one: how much of the past is kept. Δ_k , B_k and C_k are computed from the input itself, which is what makes the scan *selective* [14]. When $\Delta_k \rightarrow 0$ the state is kept and the input ignored; when Δ_k is large the past is forgotten. CFFM-Net multiplies Δ_k by the reliability of the camera that produced token k , so a token from an unreliable camera cannot write into the state.

4.3 What to learn

- Continuous state-space models and their discretisation by zero-order hold.
- The S4 view [15]: the same model as a recurrence and as a long convolution.
- Selectivity in Mamba [14], the parallel scan, and why it is linear in sequence length.
- Mamba-2 and the link between state-space models and linear attention [16].
- Applying a one-dimensional scan to an image: scan orders and directions [17, 18].
- The sceptical view [19]: when a gated convolution does as well, and why the project includes exactly that control.
- Hybrids and detectors [20, 21].

4.4 Lectures and reading

resource	link and what to take from it
The Annotated S4, Sasha Rush	https://srush.github.io/annotated-s4/ The gentlest rigorous route in, with runnable code.
A Visual Guide to Mamba	https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mamba-and-state Pictures for every step from RNNs to the selective scan.
Official Mamba code	https://github.com/state-spaces/mamba The reference implementation and its fused kernel.
Stanford CS25, Transformers United	https://web.stanford.edu/class/cs25/ Guest lectures that place state-space models next to attention.

4.5 Papers

paper	take away
S4 [15]	structured state spaces for very long sequences
Mamba [14]	input-dependent Δ , B , C : the selective scan
Mamba-2 [16]	state-space duality with attention
Vision Mamba [17]	bidirectional scans over image patches
VMamba [18]	four-direction cross scan, used by CFFM-Net
MambaOut [19]	the gated-convolution control
MambaVision [20]	a hybrid with a non-scan local branch, also used here
Mamba YOLO [21]	a scan inside a YOLO detector

4.6 In this repository

Read `cffm-net-pilot/src/cffm/blocks.py`: `GatedCrossScan` and `CMFM`. Then notebook 06, which shows numerically that a zero-reliability camera cannot change the visible outputs of the scan.

4.7 Exercises

1. Implement the recurrence above as a Python loop and as a parallel prefix scan; check they agree.
2. Show algebraically why $\Delta \rightarrow 0$ leaves the state unchanged.
3. Run the gated-convolution ablation in notebook 08 and argue, from its result, whether the scan earns its cost.

4.8 You are ready when

You can derive the discretised recurrence from the continuous model, explain what selectivity adds to S4, and point to the line in `blocks.py` where reliability enters the scan.

Stage four: seeing in two spectra

5.1 Why it matters

A visible camera sees colour and texture but goes blind at night, in glare and in smoke; a thermal camera sees heat in any light but loses texture and colour. The question the project asks is how to combine them so the result is better than either alone, and stays good when one of them fails. This stage covers the data, the fusion designs that came before CFFM-Net, and how they are evaluated.

5.2 What to learn

- What a long-wave infrared image measures, and why people and engines stand out in it.
- Registration: why the two cameras must be aligned, and what a few pixels of misalignment does to a detector.
- Early, middle and late fusion; where CFFM-Net fuses and why.
- Attention-based fusion [22, 23] and Mamba-based fusion [24–26].
- The datasets: KAIST [27], LLVIP [28], M3FD [29] and RGBT-Tiny [30]; what each is good for and what each hides.
- Robustness testing: dropping, darkening or shifting one camera at test time, and modality dropout at training time.

5.3 Papers

paper	take away
CFT [22]	cross-modality fusion with transformers
ICAFusion [23]	iterative cross-attention, a strong attention baseline
Fusion-Mamba [24]	the first Mamba fusion detector
CFMW [25]	Mamba fusion under adverse weather, one of the four source papers
COMO [26]	cross-Mamba interaction with offset-guided alignment
KAIST [27]	the original paired pedestrian benchmark, with video
LLVIP [28]	low-light paired pedestrians, the pilot's main dataset
M3FD [29]	six classes in mixed scenes, the pilot's second dataset
RGBT-Tiny [30]	tiny objects in paired video, the target benchmark

5.4 In this repository

Read `cffm-net-pilot/src/cffm/data.py`, which turns the raw datasets into paired four-channel images, then notebooks 03 and 16. The camera-failure board in notebook 16 is the clearest picture of what fusion should and should not do.

5.5 Exercises

1. Load ten LLVIP pairs, overlay visible and thermal at 50 per cent, and judge the registration by eye.
2. Shift the thermal image by 8 pixels and measure how much the concat baseline's detections move.
3. From notebook 09, explain why concat collapses when told that thermal is gone while CFFM-Net does not.

5.6 You are ready when

You can say which camera should win in a given scene and why, and you can design a test that would show whether a fusion model really behaves that way.

Stage five: tracking and video

6.1 Why it matters

A detector answers *what is here*; a tracker answers *who is this, and where are they going*. The showcase already tracks crowds, and the project’s next phase moves from still images to video, where CFEM-Net’s temporal state memory carries information from frame to frame. This stage covers the tracking-by-detection pipeline and how it is measured.

6.2 What to learn

- Tracking by detection: detect in every frame, then associate detections into identities.
- The Kalman filter as a constant-velocity motion model, and the Hungarian algorithm for matching [31].
- Using low-confidence detections to keep tracks through occlusion [32].
- Fixing drift and camera motion [33, 34].
- Metrics: MOTA, IDF1 and HOTA, and the MOT benchmarks [35].
- Visible–thermal tracking data [36] and spatio-temporal fusion [37].
- Why sampled frames cannot be tracked: the LLVIP test frames are seconds apart, which is why the showcase uses MOT17 video.

6.3 Lectures and reading

resource	link and what to take from it
Ultralytics tracking	https://docs.ultralytics.com/modes/track/ ByteTrack and BoT-SORT exactly as the showcase calls them.
ByteTrack code	https://github.com/ifzhang/ByteTrack The reference implementation.

6.4 Papers

paper	take away
SORT [31]	Kalman filter plus Hungarian matching, the baseline of the field
ByteTrack [32]	associate every box, including low-confidence ones
OC-SORT [33]	observation-centric correction of Kalman drift
BoT-SORT [34]	camera-motion compensation and appearance cues
MOT16 and MOT17 [35]	the benchmark behind the crowd demo
VT-MOT [36]	visible-thermal multi-object tracking at scale
MambaST [37]	Mamba fusion across spectra and time

6.5 In this repository

Read `cffm-net-pilot/src/cffm/showcase.py`, the `track` function, and `hud.py`, which draws identities, trails and heading arrows. Then `tsm.py`, the temporal state memory for the video phase.

6.6 Exercises

1. Implement a Kalman filter for one moving box and plot predicted against observed positions.
2. Track MOT17-04 with and without low-confidence association and count identity switches.
3. Explain why 72 identities over 150 frames of MOT17-04 is plausible and 115 is not.

6.7 You are ready when

You can explain how a detection in frame t is given the same identity as one in frame $t - 1$, and what goes wrong in a crowd.

From study to research

7.1 Reading the project's own results

The pilot is the best exercise in this guide, because its results are mixed and honest. With matched heads, CFFM-Net trails the simple concatenation baseline by 0.3 to 1.5 AP on clean LLVIP images, close to run-to-run noise. It holds up better when the thermal image is shifted, and when told that the thermal camera is gone it still detects, where concatenation collapses. When the thermal camera is switched off without warning, both models find no one, although the people are plainly visible. Each of these is a research question, and each can be stated as a hypothesis, tested with one controlled run, and decided by a rule written down in advance.

7.2 Two lessons from the training logs

The first full training run diverged to NaN from its second epoch. The cause was half precision: over twelve thousand tokens, the scan's inputs and outputs left float16's range. The fix was to keep the whole state-space path in float32. A later run diverged again once modality dropout forced one camera's weight to zero and the model pushed the other towards zero as well: the fusion divided by almost nothing, the gradient became a million times too large, and float16 overflowed. The fix was to compute the weights in float32 and floor their total. Both bugs were invisible in the unit tests and obvious in the logs. Read the logs.

7.3 A weekly rhythm

day	activity
Monday	one lecture and its notes
Tuesday	the matching exercise or assignment
Wednesday	one paper, read three times
Thursday	reproduce one figure or number from that paper, however small
Friday	read the matching code in this repository and change one thing
Weekend	write a one-page summary of the week in your own words

7.4 When you are stuck

Shrink the problem until it is small enough to see: one image instead of a dataset, one block instead of a network, ten steps instead of thirty epochs. Make a model overfit a single batch before training it properly. Change one thing at a time. When a result surprises you, suspect the code before the theory, and the data before the code.

Glossary

term	meaning
AP50, AP50-95	average precision with a box counted correct at IoU 0.5, or averaged over thresholds from 0.5 to 0.95
backbone, neck, head	feature extractor, multi-scale feature mixer, and the layer that predicts boxes and classes
ByteTrack	a tracker that associates low-confidence detections too, keeping identities through occlusion
concat baseline	two backbones whose features are stacked and squeezed by a 1x1 convolution
Δ (delta)	the step size of a selective scan; small keeps the state, large overwrites it
float16, float32	half and single precision; float16 overflows above 65504 and underflows below about 6×10^{-8}
IoU	intersection over union of two boxes
LWIR	long-wave infrared, the band a thermal camera records
modality dropout	blacking out one camera at random during training so the model learns to work without it
NMS	non-maximum suppression, removing duplicate boxes; YOLO26 does not need it

reliability gate	CFFM-Net's per-location trust in each camera, multiplied into the scan's step size
selective scan	a state-space recurrence whose parameters depend on the input
stride	how many input pixels one feature-map cell covers; stride 4 sees small objects best
trust map	a picture of the reliability gate: where the model weighted visible and where thermal

Bibliography

- [1] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015. arXiv:1412.6980.
- [2] S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *ICML*, 2015. arXiv:1502.03167.
- [3] K. He, X. Zhang, S. Ren and J. Sun. Deep residual learning for image recognition. In *CVPR*, 2016. arXiv:1512.03385.
- [4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin. Attention is all you need. In *NeurIPS*, 2017. arXiv:1706.03762.
- [5] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit and N. Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021. arXiv:2010.11929.
- [6] S. Ren, K. He, R. Girshick and J. Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *NeurIPS*, 2015. arXiv:1506.01497.
- [7] J. Redmon, S. Divvala, R. Girshick and A. Farhadi. You only look once: Unified, real-time object detection. In *CVPR*, 2016. arXiv:1506.02640.
- [8] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie. Feature pyramid networks for object detection. In *Proc. CVPR*, 2017.
- [9] T.-Y. Lin, P. Goyal, R. Girshick, K. He and P. Dollár. Focal loss for dense object detection. In *ICCV*, 2017. arXiv:1708.02002.
- [10] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov and S. Zagoruyko. End-to-end object detection with transformers. In *ECCV*, 2020. arXiv:2005.12872.

- [11] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding. YOLOv10: Real-time end-to-end object detection. In *Advances in NeurIPS*, 2024. arXiv:2405.14458.
- [12] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, and J. Chen. DETRs beat YOLOs on real-time object detection. In *Proc. CVPR*, 2024.
- [13] J. Redmon and A. Farhadi. YOLOv3: An incremental improvement. *arXiv preprint arXiv:1804.02767*, 2018.
- [14] A. Gu and T. Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- [15] A. Gu, K. Goel, and C. Ré. Efficiently modeling long sequences with structured state spaces. In *Proc. ICLR*, 2022.
- [16] T. Dao and A. Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In *Proc. Int. Conf. Machine Learning (ICML)*, 2024.
- [17] L. Zhu, B. Liao, Q. Zhang, X. Wang, W. Liu, and X. Wang. Vision Mamba: Efficient visual representation learning with bidirectional state space model. In *Proc. ICML*, 2024. arXiv:2401.09417.
- [18] Y. Liu, Y. Tian, Y. Zhao, H. Yu, L. Xie, Y. Wang, Q. Ye, and Y. Liu. VMamba: Visual state space model. In *Advances in NeurIPS*, 2024. arXiv:2401.10166.
- [19] W. Yu and X. Wang. MambaOut: Do we really need Mamba for vision? In *Proc. CVPR*, 2025. arXiv:2405.07992.
- [20] A. Hatamizadeh and J. Kautz. MambaVision: A hybrid Mamba–Transformer vision backbone. In *Proc. CVPR*, 2025. arXiv:2407.08083.
- [21] Z. Wang et al. Mamba YOLO: A simple baseline for object detection with state space model. In *Proc. AAAI Conf. Artificial Intelligence*, 2025. arXiv:2406.05835.
- [22] F. Qingyun, H. Dapeng, and W. Zhaokui. Cross-modality fusion transformer for multi-spectral object detection. *arXiv preprint arXiv:2111.00273*, 2021.
- [23] J. Shen et al. ICAFusion: Iterative cross-attention guided feature fusion for multispectral object detection. *Pattern Recognition*, 2024. arXiv:2308.07504.
- [24] W. Dong et al. Fusion-Mamba for cross-modality object detection. *IEEE Trans. Multimedia*, vol. 27, 2025. arXiv:2404.09146.
- [25] H. Li, Q. Hu, B. Zhou, Y. Yao, J. Lin, K. Yang, and P. Chen. CFMW: Cross-modality fusion Mamba for robust object detection under adverse weather. *IEEE Trans. Circuits and Systems for Video Technology*, 35(12):12066–12081, 2025. doi:10.1109/TCSVT.2025.3587918.

- [26] COMO: Cross-Mamba interaction and offset-guided fusion for multimodal object detection. *Information Fusion*, vol. 125, 2026. *arXiv preprint* arXiv:2412.18076, 2024.
- [27] S. Hwang, J. Park, N. Kim, Y. Choi, and I. S. Kweon. Multispectral pedestrian detection: Benchmark dataset and baseline. In *Proc. CVPR*, 2015.
- [28] X. Jia, C. Zhu, M. Li, W. Tang, and W. Zhou. LLVIP: A visible-infrared paired dataset for low-light vision. In *Proc. ICCV Workshops*, 2021.
- [29] J. Liu, X. Fan, Z. Huang, G. Wu, R. Liu, W. Zhong, and Z. Luo. Target-aware dual adversarial learning and a multi-scenario multi-modality benchmark to fuse infrared and visible for object detection. In *Proc. CVPR*, 2022.
- [30] X. Ying et al. Visible-thermal tiny object detection: A benchmark dataset and baselines. *IEEE Trans. Pattern Analysis and Machine Intelligence*, 2025. arXiv:2406.14482.
- [31] A. Bewley, Z. Ge, L. Ott, F. Ramos and B. Upcroft. Simple online and realtime tracking. In *ICIP*, 2016. arXiv:1602.00763.
- [32] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang. ByteTrack: Multi-object tracking by associating every detection box. In *Proc. ECCV*, 2022.
- [33] J. Cao, J. Pang, X. Weng, R. Khirodkar, and K. Kitani. Observation-centric SORT: Rethinking SORT for robust multi-object tracking. In *Proc. CVPR*, 2023.
- [34] N. Aharon, R. Orfaig, and B.-Z. Bobrovsky. BoT-SORT: Robust associations multi-pedestrian tracking. *arXiv preprint* arXiv:2206.14651, 2022.
- [35] A. Milan, L. Leal-Taixé, I. Reid, S. Roth, and K. Schindler. MOT16: A benchmark for multi-object tracking. *arXiv preprint* arXiv:1603.00831, 2016.
- [36] Visible-thermal multiple object tracking: Large-scale video dataset and progressive fusion approach (VT-MOT). *Pattern Recognition; arXiv preprint* arXiv:2408.00969, 2024.
- [37] MambaST: A plug-and-play cross-spectral spatial-temporal fuser for efficient pedestrian detection. In *Proc. IEEE Int. Conf. Intelligent Transportation Systems (ITSC)*, 2024. arXiv:2408.01037.